

Neil Farbotnik

SER321

<https://github.com/Farbotnik/ser321-spring26-C-campus/tree/main>

Part I - Using WSL Ubuntu on Windows

1. mkdir cli_assignment
2. cd cli_assignment/
3. touch stuff.txt
4. cat <<EOF>stuff.txt
5. cat -n stuff.txt
6. cat >> stuff.txt
7. mkdir draft
8. mv stuff.txt draft
9. cd draft/
 - 9.1. touch .secret.txt
10. cp -r draft final
11. mv draft draft.remove
12. mv draft.remove final
13. ls -all -recursive
14. zcat NASA_access_log_Aug95.gz
15. gunzip NASA_access_log_Aug95.gz
16. mv NASA_access_log_Aug95 logs.txt
17. mv logs.txt cli_assignment/
18. head -n 100 logs.txt
19. head -n 100 logs.txt > logs_top_100.txt
20. tail -n 100 logs.txt
21. tail -n 100 logs.txt > logs_bottom_100.txt
22. cat logs_top_100.txt logs_bottom_100.txt > logs_snapshot.txt
23. echo "Farbotnik:This is a great assignment 20260128" >> logs_snapshot .txt
24. less logs.txt
25. cut -d'%' -f1 ./marks.csv | tail -n +2 > names.txt
26. cut -d'%' -f4 marks.csv | tail -n +2 | sort -n
27. awk -F'%' '{s+= \$3} END {print s/(NR-1)}' marks.csv
28. awk -F'%' '{s+= \$3} END {print s/(NR-1)}' marks.csv > done.txt
29. mv done.txt ./final

30. mv ./final/done.txt ./final/average.txt

Part 2

1) Deadlock

```
PS C:\Users\Neil\0-code\Git\ser321examples\Threads\Deadlock> gradle run

> Task :run
Alphonse: Gaston has bowed to me!
Gaston: waiting to bow back
Gaston: Alphonse has bowed to me!
Alphonse: waiting to bow back
<=====----> 75% EXECUTING [51s]
> :run
|
```

- 2)
- 3) Each action locks the object it belongs to, so only one thread can use it at a time. Here, both threads grab one lock and then wait for the other to be released, so neither can move forward, simulated thread deadlock.

1) JMS – Hello World

```
BUILD SUCCESSFUL in 4s
2 actionable tasks: 2 executed
PS C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld> gradle run

> Task :HelloWorld:runHelloWorld
INFO | Using Persistence Adapter: KahaDBPersistenceAdapter[C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld\activemq-data\localhost\KahaDB]
INFO | JMX consoles can connect to service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi
INFO | PLISTore:[C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld\activemq-data\localhost\tmp_storage] started
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:1) is starting
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:1) started
INFO | For help or more information please see: http://activemq.apache.org
INFO | Connector vm://localhost started
Sent message: 1625291991 : Thread-1
Sent message: 1077098098 : Thread-0
Received: Hello world! From: Thread-1 : 866557142
INFO | Connector vm://localhost stopped
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:1) is shutting down
INFO | PLISTore:[C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld\activemq-data\localhost\tmp_storage] stopped
INFO | Stopping async queue tasks
INFO | Stopping async topic tasks
INFO | Stopped KahaDB
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:1) uptime 0.433 seconds
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:1) is shutdown
INFO | Using Persistence Adapter: KahaDBPersistenceAdapter[C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld\activemq-data\localhost\KahaDB]
INFO | JMX consoles can connect to service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi
INFO | KahaDB is version 6
INFO | PLISTore:[C:\Users\Neil\0-code\Git\ser321examples\Middleware\JMS\HelloWorld\activemq-data\localhost\tmp_storage] started
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:2) is starting
INFO | Apache ActiveMQ 5.15.8 (localhost, ID:DESKTOP-FM61RP8-64124-1770087378482-0:2) started
INFO | For help or more information please see: http://activemq.apache.org
INFO | Connector vm://localhost started
Sent message: 223922035 : Thread-11
Sent message: 585635409 : Thread-9
Received: Hello world! From: Thread-9 : 682862027
Received: Hello world! From: Thread-11 : 1505403552
```

- 2)
- 3) This looks like a test program that sends messages back and forth using a shared queue. A few threads push messages in, other threads pull them out, and the console prints what was sent and received. It's basically showing that messages can move between parts of a program without them being directly connected.

1) GroupJson

```
BUILD SUCCESSFUL in 746ms
5 actionable tasks: 5 executed
PS C:\Users\Neil\0-code\Git\ser32lexamples\Serialization\GroupJson> gradle run

> Task :run
Group: Administration has users: Tim, Joe, Sue,
Administration group as json string: {"Joe":{"userPwd":"joeWord","userId":"Joe"},"Sue":{"userPwd":"sueWord","userId":"Sue"},"name":"Administration","Tim":{"userPwd":"timWord","userId":"Tim"}}
Done exporting group in json to admin.json
Importing group from admin.json
names are: Joe, Sue, name, Tim,
Group: Administration has users: Joe, Sue, Tim,
Used Java serialization of the group to admin.ser
Done importing the group from admin.ser as:
Group: Administration has users: Tim, Joe, Sue,
```

- 2)
- 3) This program seems to be testing different ways to save and reload data. It builds a small group of users, turns that group into a JSON text file, reads it back in, then does the same thing using Java's built-in binary serialization. It seems to be verifying that the data survives being written to disk and loaded again in different formats.

Part 2.4: <https://app.screencast.com/FjLlsnGXGnewT>

```
[ec2-user@ip-172-31-14-155 Sockets]$ cd JavaSimpleSock2
[ec2-user@ip-172-31-14-155 JavaSimpleSock2]$ ls
README.md  build  build.gradle  src
[ec2-user@ip-172-31-14-155 JavaSimpleSock2]$ gradle SocketServer
Starting a Gradle Daemon (subsequent builds will be faster)

> Task :SocketServer
Server ready for 3 connections
Server waiting for a connection
Received the String This message works for the assignment :)
Received the Integer 100
Server waiting for a connection
Received the String This message works for the assignment :)
Received the Integer 8888
Server waiting for a connection
<=====----> 75% EXECUTING [4m 54s]
> :SocketServer
```

```
PS C:\Users\Neil\0-code\ser32lexamples-master\ser32lexamples-master\Sockets\JavaSimpleSock2> gradle SocketClient -Phost="3.148.182.86" -Pmessage="This message works for the assignment :)"
Starting a Gradle Daemon (subsequent builds will be faster)

> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///C:/Users/Neil/0-code/ser32lexamples-master/ser32lexamples-master/Sockets/JavaSimpleSock2/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

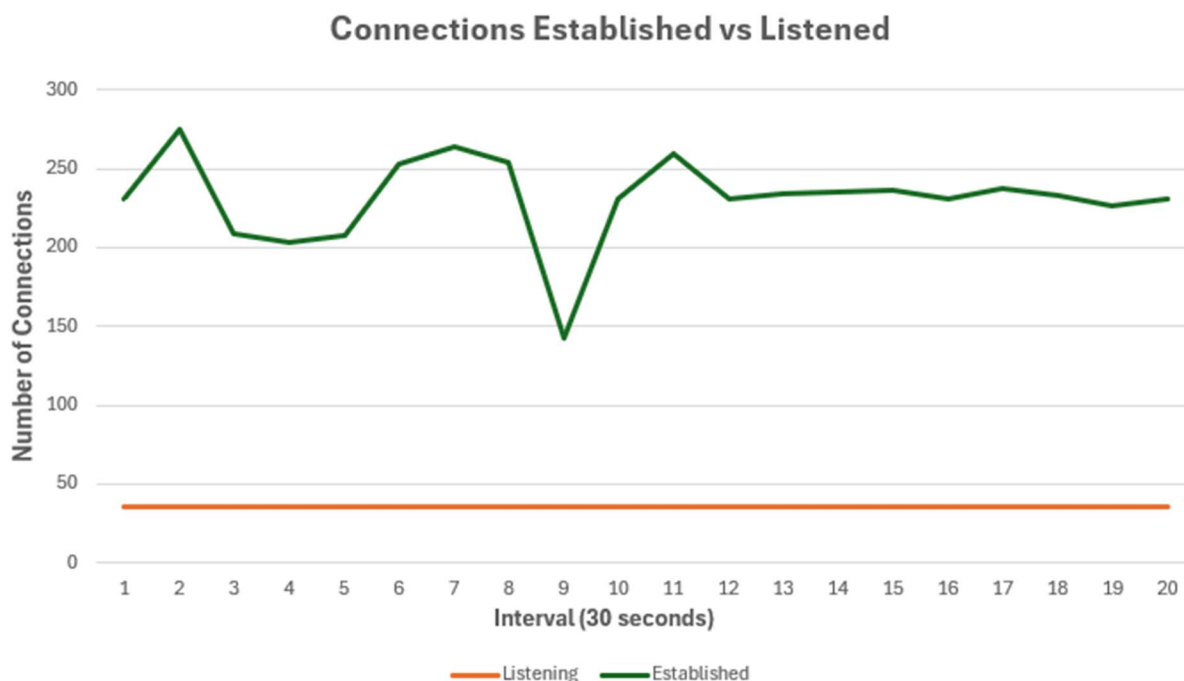
You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.13/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 13s
2 actionable tasks: 1 executed, 1 up-to-date
PS C:\Users\Neil\0-code\ser32lexamples-master\ser32lexamples-master\Sockets\JavaSimpleSock2> gradle SocketClient -Phost="3.148.182.86" -Pmessage="This message works for the assignment :)" -Pnumber=8888

> Task :SocketClient
Got it!
```

Part 3.1 Connections over 10 mins



Part 3.2

- a) `ncat -k -l 3333` to start a TCP server that listens on port 3333 and stays active after clients disconnect. In another terminal, `ncat 127.0.0.1 3333` to connect as a client to the server on the same machine. Type messages in either terminal to send data back and forth over the TCP connection.
- b) 3 packets
- c) 9 packets
- d) 22 bytes
- e) 707 bytes

f)

No.	Time	Source	Destination	Protocol	Length	Info
356	343.543691	127.0.0.1	127.0.0.1	TCP	56	53968 → 3333 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
357	343.543748	127.0.0.1	127.0.0.1	TCP	56	3333 → 53968 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
358	343.543765	127.0.0.1	127.0.0.1	TCP	44	53968 → 3333 [ACK] Seq=1 Ack=1 Win=65280 Len=0
375	348.440562	127.0.0.1	127.0.0.1	TCP	52	3333 → 53968 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=8
376	348.440577	127.0.0.1	127.0.0.1	TCP	44	53968 → 3333 [ACK] Seq=1 Ack=9 Win=65280 Len=0
381	353.289974	127.0.0.1	127.0.0.1	TCP	57	3333 → 53968 [PSH, ACK] Seq=9 Ack=1 Win=65280 Len=13
382	353.289988	127.0.0.1	127.0.0.1	TCP	44	53968 → 3333 [ACK] Seq=1 Ack=22 Win=65280 Len=0
385	355.117591	127.0.0.1	127.0.0.1	TCP	44	3333 → 53968 [FIN, ACK] Seq=22 Ack=1 Win=65280 Len=0
386	355.117608	127.0.0.1	127.0.0.1	TCP	44	53968 → 3333 [ACK] Seq=1 Ack=23 Win=65280 Len=0
387	355.117656	127.0.0.1	127.0.0.1	TCP	44	53968 → 3333 [FIN, ACK] Seq=1 Ack=23 Win=65280 Len=0
388	355.117671	127.0.0.1	127.0.0.1	TCP	44	3333 → 53968 [ACK] Seq=23 Ack=2 Win=65280 Len=0

UDP

- a) `ncat -k -l -u 3333` to start a UDP listener on port 3333 that remains active after receiving messages. In another terminal, run `ncat -u 127.0.0.1 3333` to send UDP packets to the listener on the same machine. Type messages in the client terminal to transmit datagrams, which are received without connection setup or delivery guarantees.
- b) 2 packets
- c) 2 packets
- d) 16 bytes
- e) 80 bytes
- f) 80 compared to 16 bytes
- g) TCP has higher overhead than UDP because it share additional information to manage reliable communication. TCP performs a handshake to establish a connection and uses a formal process when the connection ends, while UDP has neither. Each TCP packet includes sequence numbers, flags, and flow-control data, which increase header size, shown below. TCP also sends acknowledgment packets and may retransmit data when packets are lost, creating extra network traffic. UDP does not acknowledge packets or guarantee delivery, so its packets contain less overhead and result in lower total data transmission.

```
[TCP Segment Len: 0]
Sequence Number: 2083    (relative sequence number)
Sequence Number (raw): 3798797076
[Next Sequence Number: 2083    (relative sequence number)]
Acknowledgment Number: 52    (relative ack number)
Acknowledgment number (raw): 1592159367
0101 .... = Header Length: 20 bytes (5)
> Flags: 0x010 (ACK)
```

UDP:

udp.port==3333						
No.	Time	Source	Destination	Protocol	Length	Info
98	78.396734	127.0.0.1	127.0.0.1	UDP	40	58279 → 3333 Len=8
108	83.340812	127.0.0.1	127.0.0.1	UDP	40	58279 → 3333 Len=8

Part 3.3.1

<https://app.screencast.com/xzGiqIKkF6RaJ>

Part 3.3.2

tcp.port==8888						
No.	Time	Source	Destination	Protocol	Length	Info
2095	138.510623	192.168.0.117	3.148.182.86	TCP	66	33231 → 8888 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2098	138.585312	3.148.182.86	192.168.0.117	TCP	66	8888 → 33231 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=128
2099	138.585377	192.168.0.117	3.148.182.86	TCP	54	33231 → 8888 [ACK] Seq=1 Ack=1 Win=65280 Len=0
2100	138.587285	192.168.0.117	3.148.182.86	TCP	58	33231 → 8888 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=4
2101	138.661350	3.148.182.86	192.168.0.117	TCP	60	8888 → 33231 [ACK] Seq=1 Ack=5 Win=62848 Len=0
2102	138.661364	192.168.0.117	3.148.182.86	TCP	169	33231 → 8888 [PSH, ACK] Seq=5 Ack=1 Win=65280 Len=115
2107	138.737287	3.148.182.86	192.168.0.117	TCP	60	8888 → 33231 [ACK] Seq=1 Ack=120 Win=62848 Len=0
2108	138.750618	3.148.182.86	192.168.0.117	TCP	60	8888 → 33231 [PSH, ACK] Seq=1 Ack=120 Win=62848 Len=4
2109	138.796855	192.168.0.117	3.148.182.86	TCP	54	33231 → 8888 [ACK] Seq=120 Ack=5 Win=65280 Len=0
2111	138.871421	3.148.182.86	192.168.0.117	TCP	64	8888 → 33231 [PSH, ACK] Seq=5 Ack=120 Win=62848 Len=10
2112	138.872254	192.168.0.117	3.148.182.86	TCP	54	33231 → 8888 [FIN, ACK] Seq=120 Ack=15 Win=65280 Len=0
2132	138.990284	3.148.182.86	192.168.0.117	TCP	60	8888 → 33231 [ACK] Seq=15 Ack=121 Win=62848 Len=0

```
BUILD SUCCESSFUL in 4s
2 actionable tasks: 1 executed, 1 up-to-date
PS C:\Users\Neil\0-code\Git\ser321examples\Sockets\JavaSimpleSock2> gradle SocketClient -Phost="3.148.182.86" -Pmessage="NEW MESSAGE TO AWS ... BEEEP...BEEP" -Pnumber=8888

> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///C:/Users/Neil/0-code/Git/ser321examples/Sockets/JavaSimpleSock2/bu
```

```
> Task :SocketServer
Server ready for 3 connections
Server waiting for a connection
Received the String NEW MESSAGE TO AWS ... BEEEP...BEEP
Received the Integer 8888
Server waiting for a connection
<=====----> 75% EXECUTING [1m 46s]
> :SocketServer
```

i-00b3137270de8ec88 (SER321)

PublicIPs: 3.148.182.86 PrivateIPs: 172.31.14.155 |

When running the SockServer locally, both the client and server run on the same machine using the loopback address, so all communication stays on the local system. Capturing traffic on the loopback interface shows both the data sent by the client and the responses sent by the server. When running the SockServer on a remote cloud instance, the traffic travels over the network between my computer and the instance's public address, which changes the observed traffic. In this case, I had to capture packets on my active network interface and change the command to use the server address given by AWS (public IPv4 address) instead of localhost.

Part 3.3.3

Running the server on a home computer and the remote AWS client does not work if the client uses localhost, because localhost always refers to the machine the program is running on. In this case, the client on the cloud machine would attempt to connect to itself instead of the home computer. To make the connection work, the client must use

the public internet address of the home computer and the correct port number. The home network must also allow incoming connections, which may require port forwarding on the router and allowing the port through the local firewall. This setup is different from running both programs locally because the communication now crosses the public network instead of staying within a single machine.

Part 3.3.4

Computers inside a home network use private internet addresses that only work within that local network, while the router uses a single public internet address to communicate with the outside world. When a client on a home network connects to a server on AWS, the router allows the outgoing connection and automatically routes the response back, so the connection works easily. The opposite direction is harder because outside computers cannot directly see or reach a server that is inside a home network, since the router blocks incoming connections by default. To reach a local server from outside the network, the router must be setup to forward a specific incoming port to the correct computer inside the network, and the computer must allow that traffic.